ADVANCED COMPUTER NETWORKS · TEAM 7 · MID-PROJECT EVALUATION

Streaming a live lecture over two weak internet connections

Send whole video frames, not scattered packets, over two paths at once

Multipath QUIC · emulated cellular links · 2 November 2026

1 · THE PROBLEM

A teacher’s live video needs a steady upload that one phone often cannot give

2.6 Mbit/s

steady upload needed for a 720p class video (Zoom’s figure)

31–44%

of mobile tests in India miss a consistent-quality bar, though average uploads look fine

2 links

are often available: WiFi + mobile data, or two phones

Sources: Zoom bandwidth requirements; Opensignal India, Feb 2026: 56–69% of tests on the three private operators met its bar

2 · WHAT EXISTS TODAY

Multipath protocols split the video packet by packet

Multipath QUIC can already use two links in one connection. A **scheduler** picks a link for every packet: the faster one (MinRTT), turn by turn (round-robin), both (redundant), or by expected finish (ECF).

Commercial bonding boxes (LiveU) are closed; research that sends whole frames (MoMQ, 2026) works at a relay, not at the teacher’s phone.

The weak spot

Packets of one video frame land on both links. The frame can only be shown when its **last** packet arrives, so the slow link decides when every frame appears.

3 · OUR IDEA

Treat each video frame as one unit and place it where it finishes first

1

Predict

For each new frame, estimate when it would finish on each link.

2

Place or split

Send the whole frame on the earlier link; split a large frame across both only if that finishes sooner.

3

Rescue

If a link stalls (no progress for one timeout), resend its unfinished part on the other link.

Built inside the sender, on top of multipath QUIC: each frame part is its own QUIC stream.

4 · WHY WE THINK IT WORKS

A frame's finish time is predictable, so we can balance it

$$T = \frac{queued + frame\ size}{link\ rate} + queueing\ delay + one\text{-}way\ delay$$

Split rule

Choose the split $S_1 + S_2 = S$ so both parts finish together: $T_1(S_1) = T_2(S_2)$. One linear equation, one answer.

Worked example

Two 1.6 Mbit/s links (one-way 15 ms and 75 ms), an 85 KB frame: one link 440 ms; split 48.5 / 36.5 KB, both done at **258 ms**.

Example worked from the formula, not measured. Our 250 ms target is what makes 258 vs 440 ms matter.

5 · HOW WE TEST IT

Same video, same two connections, every scheduler

The two connections

Now: two emulated uplinks on one laptop, speed and delay set exactly (1.2–2.0 Mbit/s, 30–150 ms round trip), plus scripted 2-s outages. No phones yet.

Emulator checked with: public phone recordings replayed (Mahimahi: AT&T LTE and T-Mobile 3G, driving, USA).

Next: our own recordings, two Android phones on two different Indian operators at the same time, 3 sites.

Video

Real 60-s NPTEL blackboard lecture, encoded live-style: 2.6 Mbit/s, 25 frames/s.

Score

Frames shown on time (within 250 ms); freeze time (gaps ≥ 190 ms).

Five predictions, written down before any run

One link is often not enough (H1) · today's schedulers waste capacity when delays differ (H2) · our scheduler freezes less (H3) · it needs no special protocol (H4) · it helps most with big keyframes (H5)

6 · BUILT AND CHECKED SO FAR

The measurement setup is built and verified

Emulator is accurate

Delay within 0.4 ms, speed within 0.2%, buffer 200 ± 1 ms. Checking caught two emulator bugs, now fixed.

Multipath works

20 of 20 test transfers used both links (each 49–51% of the data).

Protocol fixes

Four small changes to the QUIC library, each with a test, incl. a connection-stall bug in the library itself.

Pipeline

Video streamer, one-command runs, scoring and statistics code, all tested.

7 · FIRST RESULTS (200 RUNS)

Two links help, but today's schedulers still drop frames

Frames shown on time (≤ 250 ms), 2.6 Mbit/s video, mean of 8 paired runs

Two links	One link	MinRTT	Round-robin	ECF
Equal: 1.6 + 1.6, 40 ms	0.4%	88.7%	**98.7%**	88.2%
Delay 30 vs 150 ms	0.4%	83.8%	**98.1%**	83.7%
Speed 2.0 vs 1.2 Mbit/s	0.8%	88.1%	**91.7%**	88.5%
2-s outages every 15 s	0.5%	66.2%	**75.2%**	31.1%

Control (4 + 4 Mbit/s): all ≈ 100%. Sending everything on both links (redundant) behaves like one link: < 1%.

Waste is real

23% spare capacity, yet MinRTT and ECF miss 11–16% of frames; they need ≈ 0.5 s delay, not 0.25 s.

Surprise

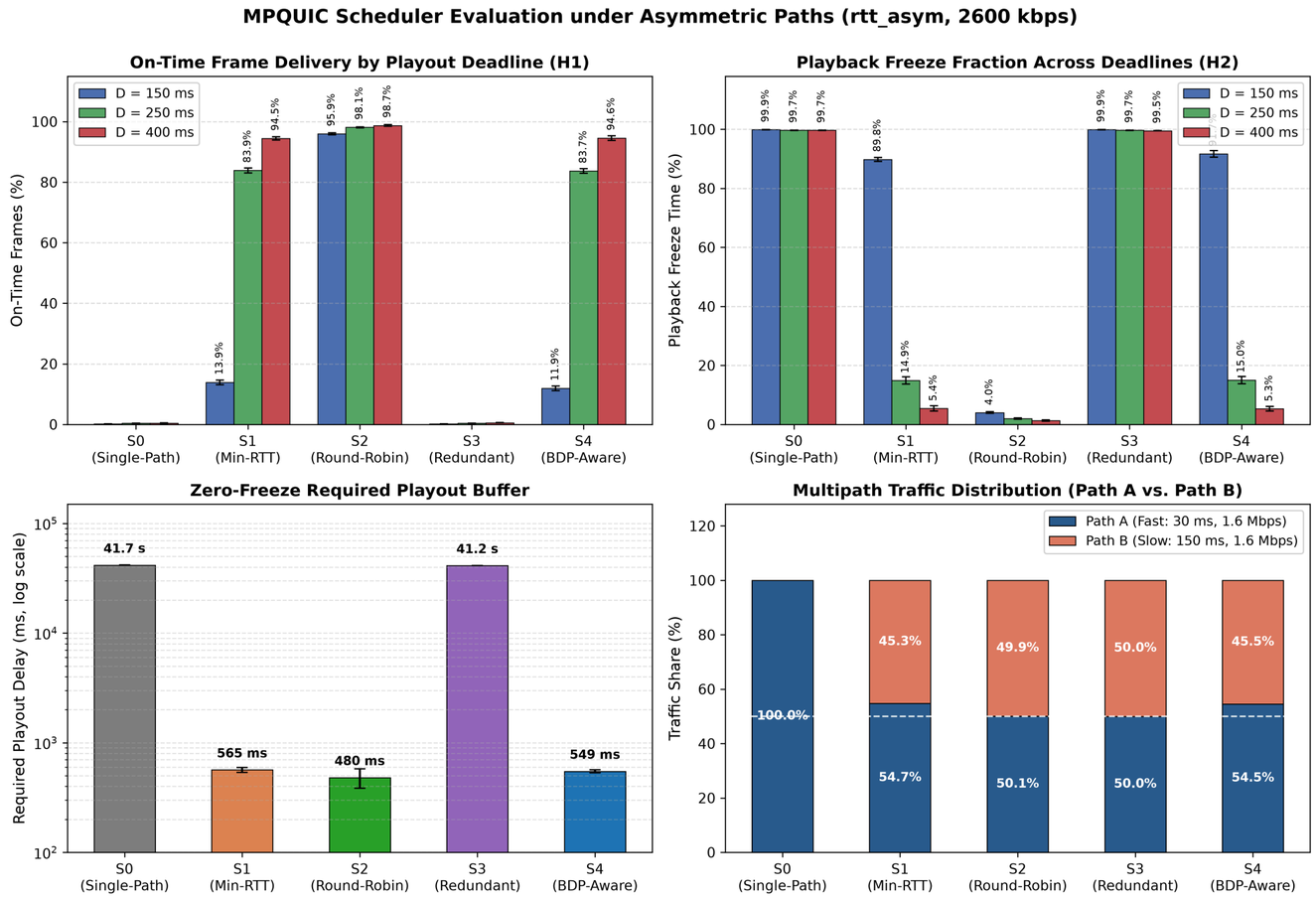
Plain round-robin wins (+3.5 to +14 points, 8 of 8 pairs). ECF collapses under outages.

Not decided yet

H2 compares with the ideal link over a bitrate sweep. One bitrate is not enough: due 11 Nov.

8 · ASYMMETRIC RTT PATH DIAGNOSIS (30 MS VS. 150 MS)

Equal path sharing prevents head-of-line queue buildup



Deadline Resilience (H1)

At tight **D = 150 ms** deadlines, Round-Robin (S2) delivers **95.9%** of frames on time with only **4.0%** freeze, vs. **13.9%** on-time (**89.8%** freeze) for MinRTT.

Why MinRTT & ECF Stall

Both paths carry **1.6 Mbit/s**, yet MinRTT (**54.7%**) and ECF (**54.5%**) overload Path A during intra-frame bursts, inflating required buffer to **549–565 ms**.

Design Takeaway

Frame-aware scheduling must respect per-path bottleneck rate rather than greedily filling the low-RTT queue.

8 · NEXT STEPS

Plan to the final demo on 17 November

By 11 Nov

Ideal-link reference; real recordings at 3 sites. Decide: is there a problem worth solving?

12–14 Nov

Build the frame scheduler; run the full comparison (~1,240 runs, overnight).

15–17 Nov

Statistics, plots, report; final demo on 17 Nov.

Main risks

Real recordings need two phones on two operators and a cloud server (tools ready and tested). The schedule is tight: about 25 hours of runs in four nights.

Thank you. Questions?

Problem: one weak uplink. Existing fix: split packets over two links. Our bet: split by whole frames, timed to finish together.

Video: Prof. V. Balakrishnan, IIT Madras, NPTEL (CC BY-SA). Stack: TQUIC (Tencent), Linux netem/htb, ffmpeg/x264.